

Lab 3: Architectural Justification

Student Project Portfolio & Showcase App

Architecture Selection: We chose a Microservices Architecture for the Student Project Portfolio & Showcase App.

Reason 1 — Independent scaling for the expo-day traffic spike

Usage is extremely uneven: profile submission happens over weeks, judge grading happens in short scheduled windows, but public voting and leaderboard traffic spike massively during the live expo. Microservices let us scale only the Voting Service and Leaderboard & Analytics Service to handle that peak, instead of over-provisioning the entire system as a single monolith would require.

Reason 2 — Fault isolation across very different actor workflows

Student Teams, Panel Judges, and the public interact with different parts of the system with different reliability needs. If the Voting Service is overloaded or degraded during peak public traffic, Team Profile submission and Judge grading must keep working. Splitting these into separate services means a failure or slowdown in one does not cascade into the others.

Security Advantage

Anti-sybil protection and vote-integrity logic are isolated inside the Voting Service, reachable only through the API Gateway's authenticated IVoteCast interface. This confines abuse-detection, rate-limiting, and duplicate-vote logic to one auditable service boundary, rather than scattering vote-validation code across the profile or judging services where it could be bypassed.

Performance Benefit

NFR-001 requires leaderboard updates under 300ms even under peak load. Because the Leaderboard & Analytics Service is a dedicated, independently deployable component receiving push-based IScoreUpdate events from both Judging and Voting Services, it can be scaled horizontally and optimized purely for fast read/aggregation workloads, without competing for resources with profile submission or grading traffic.

